

Sistemi Operativi

Corso di Laurea in Informatica

Sapienza Università di Roma

a.a. 2026/2027

Marco Raoul Marini

marini@di.uniroma1.it

Gabriele Tolomei

tolomei@di.uniroma1.it

Module 2: Virtualization

Lecture 2.1: Processes and APIs

Opening Thought

"The most fundamental abstraction that the OS provides to users in order to manage running programs — the process."

— Remzi & Andrea Arpaci-Dusseau, OSTEP

The process is not just a concept. It is the **central unit of work** around which every other OS mechanism revolves.

By the end of this lecture you will think of every running program as a structured, OS-managed entity — and you will be able to create, inspect, and control processes from C.

| Lecture Roadmap

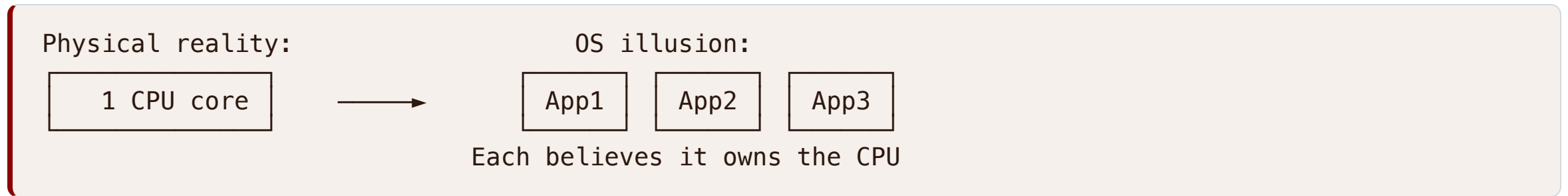
This lecture is organized into **8 major sections**:

1. **The Process Abstraction** — definition, memory layout, states
2. **Process Data Structures** — PCB, context, process list
3. **The UNIX Process API** — overview and philosophy
4. **fork()** — creating new processes
5. **wait()** and **waitpid()** — synchronizing with children
6. **exec()** — replacing the process image
7. **Other Process APIs** — **exit**, **kill**, **pipe**, **dup2**, **/proc**
8. **The Shell: Putting It All Together** — a complete example

Let's build the machine that runs programs.

| The Core Problem: CPU Virtualization

A modern computer has **one (or a few) physical CPUs**, yet appears to run **many programs simultaneously**.



The OS solution: Run one process for a short time, then another, then another. This **time-sharing** creates the illusion of parallelism.

The key abstraction that makes this possible: the **Process**.

| The OSTEP Thematic Arc

This course follows the structure of OSTEP's three themes:

VIRTUALIZATION ← We are here Make one resource look like many
CONCURRENCY Manage shared access safely
PERSISTENCE Store data reliably beyond RAM

Virtualization itself splits into **CPU virtualization** (processes + scheduling) and **memory virtualization** (address spaces, paging, swapping).

§ 1 — The Process Abstraction

| What is a Process?

A process is a running instance of a program.

More precisely, a process is the OS abstraction of a **program in execution**, including:

- The **program code** (instructions) being executed
- The current value of the **Program Counter (PC)**
- The contents of CPU **registers** (general purpose, stack pointer, flags)
- A private **virtual address space** (code, data, heap, stack)
- A set of **open file descriptors**
- A unique **Process ID (PID)**

A program on disk is inert — a recipe. A process in memory is alive — a chef cooking.

| Program vs. Process

The distinction is fundamental and constantly tested:

Aspect	Program	Process
Nature	Passive entity	Active entity
Location	Binary file on disk (ELF, EXE)	Active allocation in RAM
Lifetime	Permanent (until deleted)	Temporary (exec to exit)
State	None — static bytes	Dynamic: PC, registers, memory
Count	One file	Many processes from same binary
Resources	Disk space only	CPU, RAM, file descriptors

The same program (e.g., `/bin/ls`) can have many simultaneous processes — one per terminal session.

| How the OS Creates a Process

When you run a program, the OS performs these steps:

1. Read the ELF binary from disk

`hello (ELF)` ← binary file on disk

2. Load code and static data into RAM
(eagerly or lazily – more later)

`code (.text)` ← instructions loaded
`data (.data)` ← global variables

3. Allocate a stack and heap
4. Set up file descriptors (`stdin=0`, `stdout=1`, `stderr=2`)
5. Jump to entry point: `main()`

| Eager vs. Lazy Loading

The OS has two strategies for loading program code into RAM:

Eager Loading (Early OSes)

- Load the **entire** binary into memory before starting execution.
- Simple but wastes RAM for code that may never run.

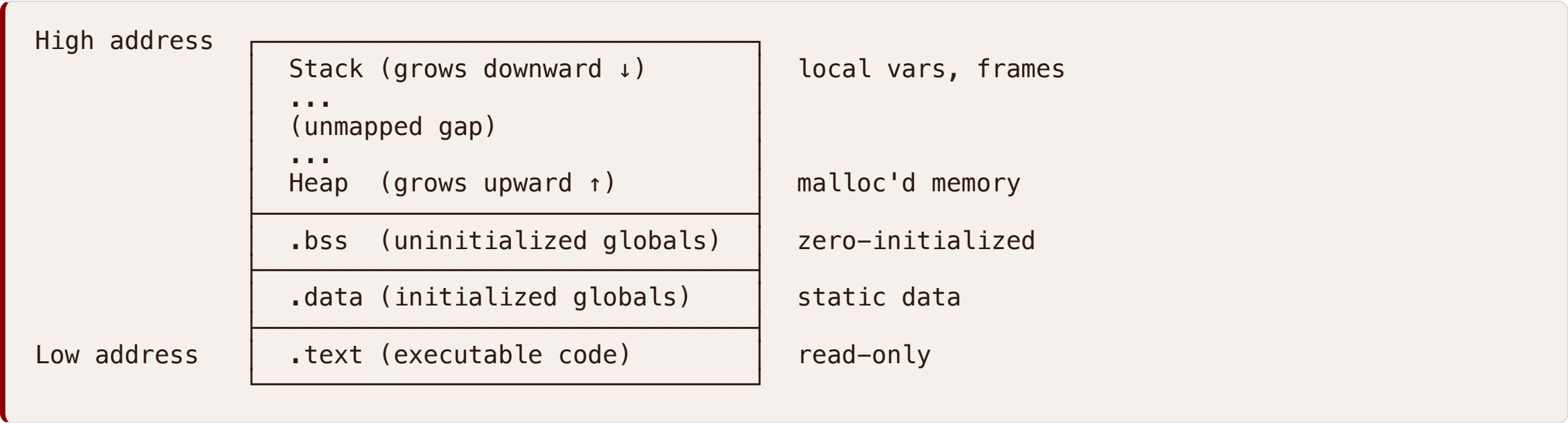
Lazy Loading (Modern OSes — On-Demand Paging)

- Load **only** what is needed, **as it is needed**.
- The OS sets up page table entries that point to disk.
- When the CPU accesses an unmapped page → **page fault** → OS loads it now.
- Most of the binary may never be loaded if those code paths are not reached.

We will study paging and page faults in depth later.

| Process Memory Layout

Every process sees a structured virtual address space:



Each process believes it owns the **entire address space** — this is the memory virtualization illusion.

| Initial Process State

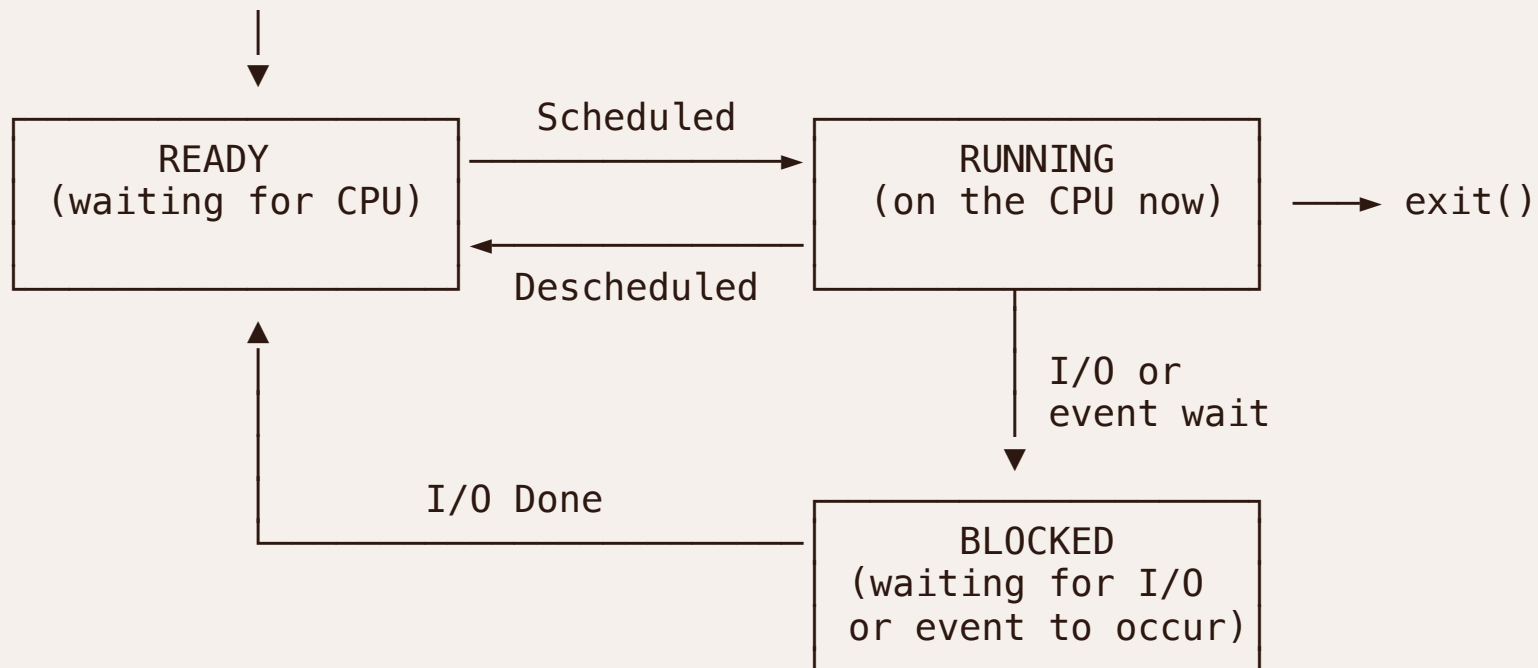
When a process first starts:

- **Stack:** contains `argc`, `argv[]`, `envp[]` (environment variables) already set up by the OS loader.
- **Heap:** empty — the program must call `malloc()` / `brk()` to grow it.
- **PC:** points to the first instruction of `main()` (after the C runtime `_start` entry point initializes libc).
- **File descriptors:** 0 (stdin), 1 (stdout), 2 (stderr) are open and connected to the terminal.

```
int main(int argc, char *argv[], char *envp[]) {  
    // argc: number of command-line arguments  
    // argv: array of argument strings (argv[0] = program name)  
    // envp: array of "KEY=value" environment strings  
}
```

Process States

A process is always in one of these core states:



| Process State: RUNNING

RUNNING — the process is currently executing on a CPU core.

- Its instructions are being fetched and decoded by the CPU.
- Its registers are loaded in the physical CPU register file.
- At any given time, only **one process per core** can be in this state.

On a 4-core machine, at most **4 processes** can truly be running simultaneously.

The OS scheduler decides which READY process gets promoted to RUNNING.

| Process State: **READY**

READY — the process is ready to run but the CPU is busy with another process.

- Its context (registers, PC) is saved in the PCB.
- It is sitting in the **run queue** (scheduler's list of runnable processes).
- The moment a CPU becomes free, the scheduler picks from the run queue.

This state is where processes spend most of their waiting time when the system is CPU-bound.

Process State: **BLOCKED**

BLOCKED — the process is waiting for an external event to complete.

Common causes of blocking:

- **Disk I/O:** `read()` from a slow HDD/SSD
- **Network I/O:** waiting for data to arrive on a socket
- **Terminal input:** `read()` waiting for the user to press Enter
- **Sleep:** `sleep(3)` — timer not yet expired
- **Lock contention:** waiting to acquire a mutex (more on this later...)

*While blocked, the process is **not runnable** — the scheduler skips it entirely.
This is essential: blocking processes should not waste CPU time polling.*

Tracing State Transitions

Let's trace two processes:

- **Process A:** CPU-bound (mostly computation)
- **Process B:** I/O-bound (performs a disk read)

Time →	1	2	3	4	5	6
CPU:	B	A	A	A	B	A
A:	Ready	Run	Run	Run	Ready	Run
B:	Run	Block	Block	Ready	Run	Done
Issues disk read Disk read completes						

Tracing State Transitions

Time →	1	2	3	4	5	6
CPU:	B	A	A	A	B	A
A:	Ready	Run	Run	Run	Ready	Run
B:	Run	Block	Block	Ready	Run	Done

Issues disk read Disk read completes

1. **Time 1:** Process **B** is running and issues a disk read.
2. **Time 2:** Because the I/O will take time, **B** transitions to **BLOCKED**. The scheduler immediately dispatches **A**.
3. **Times 2–4:** While **B** waits for the disk, **A** continues executing on the CPU.
4. **Time 4:** The disk finishes reading the data and generates an interrupt. **B** moves from **BLOCKED** to **READY**.
5. **Time 5:** The scheduler selects **B** to run again.

Key idea: The CPU executes **Process A** while the disk performs I/O for **Process B**. This overlap between computation and I/O is the fundamental idea behind **multiprogramming**: instead of leaving the CPU idle while one process waits, the operating system keeps another ready process running.

§ 2 — Process Data Structures

| The Process Control Block (PCB)

The OS maintains a **Process Control Block** for every process — its complete bookkeeping record.

```
// Simplified xv6-style PCB (struct proc):
struct proc {
    int          pid;           // Process ID
    enum procstate state;      // RUNNING, READY, BLOCKED, ZOMBIE, UNUSED
    uint64       sz;           // Size of process memory (bytes)
    pagetable_t  pagetable;    // Page table (virtual → physical)
    char         *kstack;      // Bottom of kernel stack for this process
    struct trapframe *trapframe; // Saved user registers
    struct context context;    // Saved kernel registers (for context switch)
    struct file  *ofile[NFILE]; // Open file descriptors
    struct inode *cwd;          // Current working directory
    char         name[16];      // Process name (for debugging)
};
```

| PCB Key Fields Explained

Field	Purpose
pid	Unique numeric identifier (assigned by OS at creation)
state	Current state: RUNNING / READY / BLOCKED / ZOMBIE
pagetable	Pointer to this process's page table (memory map)
trapframe	Snapshot of all CPU registers when interrupted
context	Kernel-side register state for context switching
ofile[]	Array of open file descriptor pointers
cwd	Inode of the current working directory (pwd)
name	Human-readable name (shown by ps , top)

The PCB is the OS's **complete record** of a process. Context switching = saving/restoring PCBs.

| The Saved Context

When the OS suspends a process (preemption or blocking), it saves the **CPU register state** into the PCB's `trapframe`:

Saved on interrupt/syscall into trapframe:

```
PC  (instruction pointer – where to resume)
SP  (stack pointer)
General purpose: RAX, RBX, RCX, RDX, ...
RSI, RDI, R8–R15
RFLAGS (condition flags: ZF, CF, OF, ...)
Segment registers: CS, SS
```

When the process is scheduled again, these values are **restored** to the physical CPU registers — the process resumes exactly where it left off, unaware anything happened.

| The Process List

The OS maintains a **process table** — a list/array of all PCBs:

Process Table (simplified):

PID	Name	State	Other PCB fields...
1	systemd	READY	...
42	bash	READY	...
87	firefox	BLOCKED	...
99	ls	RUNNING	...
100	(unused)	UNUSED	...

The **scheduler** scans this table looking for READY processes to run.

The **ps** and **top** commands read this table (via **/proc**) to display process info.

Process Hierarchy

On UNIX, processes form a **tree** rooted at PID 1:

```
PID 1: systemd (init)
├── PID 120: sshd
│   └── PID 201: bash (your login shell)
│       ├── PID 310: vim
│       └── PID 311: ls
├── PID 130: NetworkManager
└── PID 140: cron
```

- Every process has a **parent** (PPID — Parent PID).
- The **only** way to create a new process is via `fork()`.
- When a parent dies before its children, the children are **adopted** by PID 1 (init/systemd).

| PID 1 — The Init Process

PID 1 is special:

- The **first user-space process** started by the kernel after boot.
- On modern Linux: **systemd** (or `/sbin/init` on older systems).
- It is the **ancestor of all other processes**.
- It runs a loop that calls `wait()` to reap any orphaned child processes — preventing zombie accumulation.
- PID 1 **cannot be killed** by normal signals — `kill -9 1` is ignored by the kernel.

If PID 1 crashes, the kernel panics — there is nothing left to manage user space.

✓ Checkpoint 1 — Process Abstraction & Data Structures

Test your understanding:

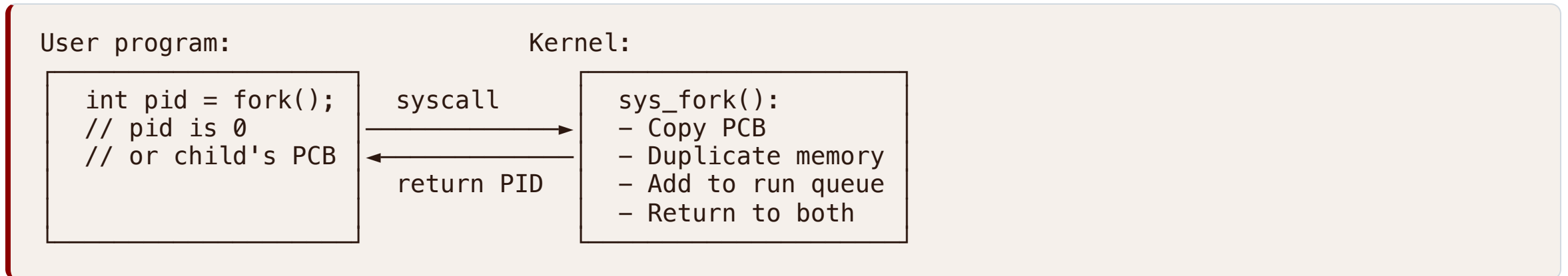
1. What is the difference between a **program** and a **process**? Give a concrete example.
2. List the **three core process states** and describe what causes transitions between them.
3. What is a **PCB**? Name four fields it contains and explain each.
4. Why does the OS save CPU register values when suspending a process?
5. What is the significance of **PID 1**, and why is it special?
6. What happens to a child process when its parent terminates before calling `wait()` ?

Write your answers before continuing.

§ 3 — The UNIX Process API

| What is a System Call (Syscall)?

The OS exposes process management through **system calls** — the boundary between user space and kernel space.



C library functions like `fork()`, `wait()`, `execvp()` are **thin wrappers** around kernel syscalls (on Linux: `SYS_clone`, `SYS_wait4`, `SYS_execve`).

| The Core UNIX Process API

Five system calls form the backbone of process management:

Syscall	Purpose
<code>fork()</code>	Create a child process (clone of the caller)
<code>exec()</code> family	Replace the current process image with a new program
<code>wait()</code> / <code>waitpid()</code>	Wait for a child to terminate; collect exit status
<code>exit()</code>	Terminate the current process with an exit code
<code>getpid()</code> / <code>getppid()</code>	Query current process ID / parent process ID

Required headers: `#include <unistd.h>`, `#include <sys/wait.h>`, `#include <sys/types.h>`

| The UNIX Design Philosophy

Why this seemingly complex API? Why not a single `spawn()` call?

"Do one thing, and do it well."

Separating `fork()` and `exec()` gives the shell **enormous flexibility**:

- Between `fork()` and `exec()`, the child can **redirect I/O** (`dup2`), **set environment variables**, **adjust signals**, **set process group** — all before the new program starts.
- This design enabled pipes (`|`), redirection (`>`, `<`), background jobs (`&`), and job control — all without changing the kernel API.

A unified `spawn()` would need hundreds of parameters to replicate this.

Required Headers & Compilation

```
#include <stdio.h>      // printf, perror
#include <stdlib.h>      // exit, EXIT_SUCCESS, EXIT_FAILURE
#include <unistd.h>      // fork, exec, getpid, getppid, pipe, dup2
#include <sys/types.h>   // pid_t
#include <sys/wait.h>    // wait, waitpid, WIFEXITED, WEXITSTATUS
#include <string.h>      // strerror
#include <errno.h>       // errno
#include <fcntl.h>       // open, O_WRONLY, O_CREAT, O_TRUNC
```

Compile with:

```
gcc -Wall -Wextra -o program program.c
```

*Always compile with **-Wall -Wextra** to catch common bugs early.*

§ 4 — `fork()`: Creating Processes

| The `fork()` Concept

`fork()` creates a **new process** — the child — by making an almost exact copy of the calling process — the parent.

```
#include <unistd.h>
pid_t fork(void);
```

After `fork()` returns:

- **Two processes exist** — parent and child.
- Both continue executing from the **same point** in the code (just after `fork()`).
- They share the same code, but have **independent copies** of all data and stack.
- The **only** difference between them is the **return value** of `fork()` .

*`fork()` is called once — but returns **twice** (once in the parent, once in the child).*

fork() Return Values

The return value is how parent and child distinguish themselves:

Context	Return Value	Meaning
Parent process	<code>> 0</code> (a positive integer)	The PID of the newly created child
Child process	<code>== 0</code>	"I am the child"
Error	<code>== -1</code>	<code>fork()</code> failed; check <code>errno</code>

```
pid_t pid = fork();

if (pid < 0) {
    perror("fork failed"); // system call failed
    exit(EXIT_FAILURE);
} else if (pid == 0) {
    // — CHILD executes this branch —
} else {
    // — PARENT executes this branch —
    // pid holds the child's PID
}
```

Minimal `fork()` Example

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int main(void) {
    printf("Before fork: PID=%d\n", getpid());

    pid_t pid = fork();

    if (pid < 0) {
        perror("fork"); exit(1);
    } else if (pid == 0) {
        // Child
        printf("Child:  PID=%d, PPID=%d\n", getpid(), getppid());
    } else {
        // Parent
        printf("Parent: PID=%d, child PID=%d\n", getpid(), pid);
    }

    printf("Both parent and child print this line.\n");
    return 0;
}
```

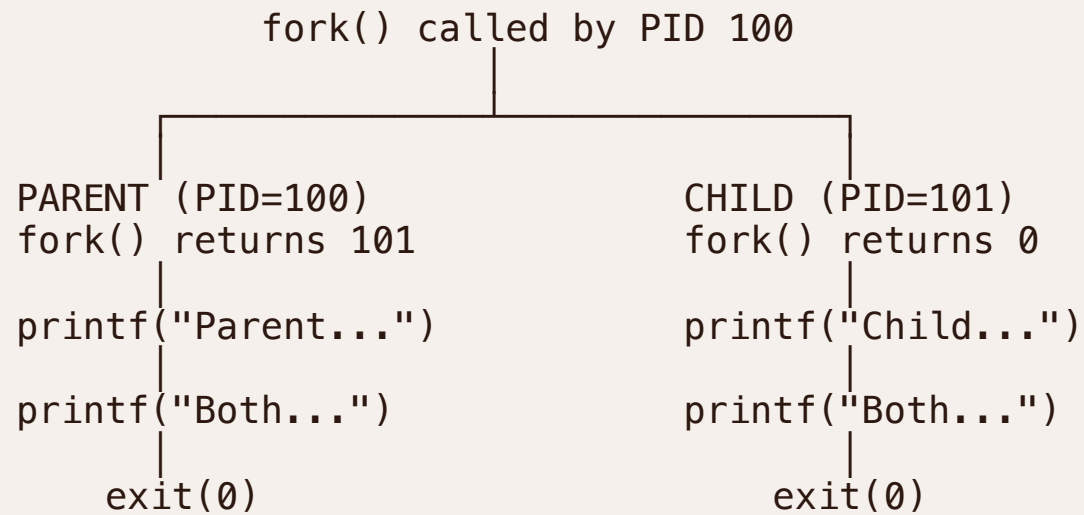
| Minimal `fork()` Example

Sample output (order may vary):

```
Before fork: PID=100
Parent: PID=100, child PID=101
Child:  PID=101, PPID=100
Both parent and child print this line.
Both parent and child print this line.
```

`fork()` Execution Flow

Visualizing what happens in memory and time:



Both parent and child run **concurrently** after the fork. The OS scheduler decides which runs first.

| Address Space After `fork()`

The child receives a **full copy** of the parent's virtual address space:

PARENT address space:

.text (code)
.data / .bss
Heap
Stack

—share→

—copy→

—copy→

—copy→

CHILD address space (copy):

.text (code)
.data / .bss
Heap
Stack

← same physical pages (COW)

← independent copy

← independent copy

← independent copy

Modifications in the child do **not** affect the parent and vice versa — they are fully independent after `fork()`.

Non-Determinism: Who Runs First?

After `fork()`, the OS scheduler decides which process — parent or child — runs first.

This is non-deterministic and depends on many factors:

- Number of CPU cores
- Current scheduler load
- OS version and scheduler algorithm

```
$ ./fork_example
Before fork: PID=100
Parent: PID=100, child PID=101    # sometimes parent first
Child:  PID=101, PPID=100

$ ./fork_example
Before fork: PID=100
Child:  PID=101, PPID=100        # sometimes child first
Parent: PID=100, child PID=101
```

Do not write code that depends on a fixed parent/child ordering. Use `wait()` to enforce synchronization.

fork() with Conditional Branching

The standard pattern uses if/else to give different behavior to parent and child:

```
#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>

int x = 100;    // global variable – each process gets its own copy

int main(void) {
    pid_t pid = fork();

    if (pid == 0) {
        // Child: modify x independently
        x = 200;
        printf("Child:  x = %d (PID=%d)\n", x, getpid());
    } else {
        // Parent: x is unaffected by child's change
        sleep(1); // let child run first
        printf("Parent: x = %d (PID=%d)\n", x, getpid());
    }
    return 0;
}

// Output:
// Child:  x = 200 (PID=101)
// Parent: x = 100 (PID=100)  ← unchanged!
```

getpid() and getppid()

Two useful functions to identify a process and its parent:

```
#include <unistd.h>
#include <sys/types.h>

pid_t getpid(void);    // returns the PID of the calling process
pid_t getppid(void);  // returns the PID of the calling process's parent
```

Usage in a fork context:

```
pid_t pid = fork();
if (pid == 0) {
    printf("I am the child.\n");
    printf("  My PID:      %d\n", getpid());
    printf("  My PPID:      %d\n", getppid());    // should match parent's getpid()
} else {
    printf("I am the parent.\n");
    printf("  My PID:        %d\n", getpid());
    printf("  Child's PID: %d\n", pid);
}
```

Multiple `fork()` Calls — How Many Processes?

```
fork(); // call 1
fork(); // call 2
fork(); // call 3
printf("Hello!\n");
```

Each `fork()` doubles the number of processes:

```
Start:    1 process
After fork #1:  2 processes (A, B)
After fork #2:  4 processes (A, B, C, D)
After fork #3:  8 processes (A, B, C, D, E, F, G, H)
```

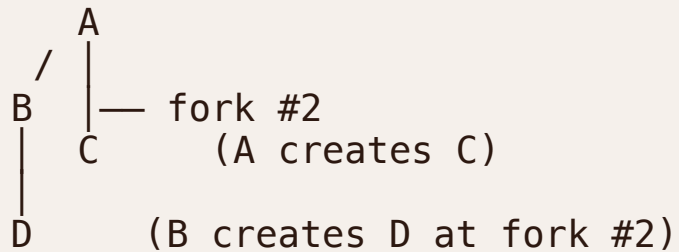
Rule: After `n` unconditional `fork()` calls → **2^n processes** exist.

All 8 processes print "Hello!" → **8 lines** of output.

Always trace a fork tree carefully when asked in an exam.

Fork Tree Diagram

```
int main(void) {           // Process A (original)
    pid_t p1 = fork();      // Creates B
    pid_t p2 = fork();      // A creates C, B creates D
}
```



Process:	A	B	C	D
p1 value:	B_pid	0	B_pid	0
p2 value:	C_pid	D_pid	0	0

Reading fork trees correctly is a core exam skill.

| The Fork Bomb

A **fork bomb** is a denial-of-service attack that exhausts the OS process table:

```
:(){ :|:& };;:      # bash fork bomb
```

Or in C:

```
while (1) {  
    fork();    // exponentially creates processes until the system runs out  
}
```

What happens:

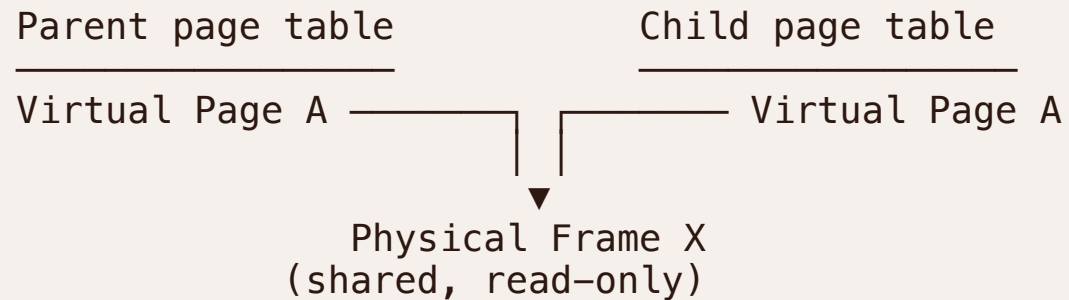
1. Each fork creates two processes, each of which forks again.
2. The number of processes grows **exponentially** (2, 4, 8, 16 ...).
3. The process table fills up → `fork()` returns `-1` for everyone.
4. Memory and CPU are saturated → system becomes unresponsive.

Defense: `ulimit -u 100` limits the maximum number of processes per user.

Copy-on-Write (COW)

Naively, `fork()` would copy the entire address space—potentially several gigabytes—every time a new process is created. Modern operating systems avoid this cost using **Copy-on-Write (COW)**.

After `fork()`:



Both processes initially **share the same physical memory pages**. Their page table entries point to the same physical frames, which are marked **read-only**.

Copy-on-Write (COW)

Parent writes to the page:

1. CPU detects a write to a read-only page
2. A page fault is generated
3. The OS allocates a new physical frame Y
4. Contents of X are copied into Y
5. Parent's page table is updated to point to Y
6. Parent resumes execution

Result:

Parent → Physical Frame Y
Child → Physical Frame X

Key idea: Memory pages are copied **only when one of the processes modifies them**. If neither process writes to a page, it remains shared.

Why it matters: Copy-on-Write makes `fork()` much more efficient by avoiding unnecessary copying of memory. The kernel duplicates the process metadata and page tables, but the actual memory pages are copied only on demand.

| What `fork()` Does NOT Inherit

Some resources are **NOT** duplicated into the child:

Resource	Behavior after <code>fork()</code>
Memory	Copied (COW) — independent
File descriptors	Shared (same underlying open-file table entries)
File locks	Not inherited
Pending signals	Not inherited
Timer alarms	Not inherited
PID / PPID	New PID assigned; PPID = parent's PID
Signal handlers	Inherited (same function pointers)
Environment variables	Inherited (full copy)

Sharing file descriptors between parent and child requires careful coordination — both can read/write the same file position.

✓ Checkpoint 2 — `fork()`

Test your understanding:

1. What does `fork()` return in the **parent** process? In the **child** process? On **error**?
2. After `fork()`, does the child start execution from the beginning of `main()`? Explain.
3. The parent sets `int x = 5`, then calls `fork()`, and the child sets `x = 99`. What does the parent see when it later reads `x`?
4. How many processes are alive after three unconditional `fork()` calls?
5. Draw the **fork tree** for: `fork(); if (fork() == 0) fork();`
6. What is **Copy-on-Write**, and why does it make `fork()` efficient?

Write and compile your own fork experiment before continuing.

§ 5 — `wait()` and `waitpid()`

The Zombie Problem

When a child process exits, it does not disappear immediately:

```
Child exits → becomes a ZOMBIE
- Code and data are freed
- PCB is kept with exit status → waiting for parent to collect it
- Shows up in `ps` as "Z" (zombie) or "<defunct>"
```

```
Parent calls wait() → ZOMBIE is fully cleaned up
```

If the **parent never calls** `wait()`, the child's PCB accumulates as a zombie, wasting OS resources.

```
$ ps aux | grep defunct
user 1234  0.0  0.0  zombie  <defunct>  ← zombie process
```

A zombie is not dangerous in small numbers, but a program that `fork()`s without `wait()`ing will eventually fill the process table.

wait() — Basic Usage

```
#include <sys/wait.h>
pid_t wait(int *wstatus);
```

- **Blocks** the calling process until **any** child terminates.
- Returns the **PID** of the child that terminated.
- Writes the child's exit status into ***wstatus** (if not NULL).
- Returns **-1** if there are no children (sets **errno = ECHILD**).

```
int status;
pid_t child_pid = wait(&status);

if (child_pid == -1) {
    perror("wait");
} else {
    printf("Child %d exited.\n", child_pid);
    if (WIFEXITED(status)) {
        printf("Exit code: %d\n", WEXITSTATUS(status));
    }
}
```

Making Output Deterministic with `wait()`

Without `wait()`, the order of parent and child output is unpredictable.

With `wait()`, the parent can force itself to wait for the child to finish:

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main(void) {
    pid_t pid = fork();
    if (pid == 0) {
        printf("Child is running (PID=%d)\n", getpid());
        exit(42);    // exit with code 42
    }
    // Parent waits for child
    int status;
    pid_t done = wait(&status);
    printf("Parent: child %d done, exit code=%d\n",
        done, WEXITSTATUS(status));
    return 0;
}

// Output (always in this order):
// Child is running (PID=101)
// Parent: child 101 done, exit code=42
```

`waitpid()` – Targeted Waiting

`wait()` waits for *any* child. `waitpid()` lets you specify **which** child to wait for:

```
#include <sys/wait.h>
pid_t waitpid(pid_t pid, int *wstatus, int options);
```

pid argument	Behavior
<code>> 0</code> (specific PID)	Wait for that exact child
<code>-1</code>	Wait for any child (same as <code>wait()</code>)
<code>0</code>	Wait for any child in the same process group
<code>< -1</code>	Wait for any child in process group <code>abs(pid)</code>

Common options:

- `0` — block until child changes state
- `WNOHANG` — return immediately (0) if no child has exited yet
- `WUNTRACED` — also return for stopped (SIGSTOP) children

waitpid() with WNOHANG

WNOHANG makes waitpid() non-blocking — useful for polling:

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main(void) {
    pid_t pid = fork();
    if (pid == 0) {
        sleep(2);    // child sleeps 2 seconds
        exit(0);
    }
    // Parent polls every 500ms
    int status;
    pid_t result;
    do {
        result = waitpid(pid, &status, WNOHANG);
        if (result == 0) {
            printf("Child not done yet, doing other work...\n");
            usleep(500000);    // sleep 0.5 seconds
        }
    } while (result == 0);

    printf("Child %d finished.\n", pid);
    return 0;
}
```

Exit Status Macros

The `wstatus` value from `wait()` / `waitpid()` is encoded. Use these macros:

Macro	Meaning
<code>WIFEXITED(status)</code>	True if child exited normally via <code>exit()</code>
<code>WEXITSTATUS(status)</code>	Extract the exit code (0–255) — only if <code>WIFEXITED</code>
<code>WIFSIGNALED(status)</code>	True if child was killed by a signal
<code>WTERMSIG(status)</code>	Signal number that killed the child
<code>WIFSTOPPED(status)</code>	True if child was stopped (SIGSTOP)

```
int status;
wait(&status);

if (WIFEXITED(status))
    printf("Exited with code %d\n", WEXITSTATUS(status));
else if (WIFSIGNALED(status))
    printf("Killed by signal %d\n", WTERMSIG(status));
```

| Orphan Processes

An **orphan** is a child whose parent dies before it does:

```
graph LR; P["Parent (PID 100) exits"] --> G["(gone)"]; C["Child (PID 101) is still running"]; C --> G; A["Adopted by PID 1"]; C --> A;
```

Parent (PID 100) exits —————→ (gone)

Child (PID 101) is still running —————→

PPID was 100, now becomes 1 (init/systemd)

Adopted by PID 1 —————→

- The kernel automatically **re-parents** orphans to PID 1.
- systemd (PID 1) runs `wait()` in a loop, so orphans are eventually reaped.
- Orphans are **not dangerous** — they are a handled edge case.

Compare: **Zombie** = child exits, parent doesn't `wait()` (PCB lingers).

Orphan = parent exits, child is still running (re-parented to init).

Waiting for Multiple Children

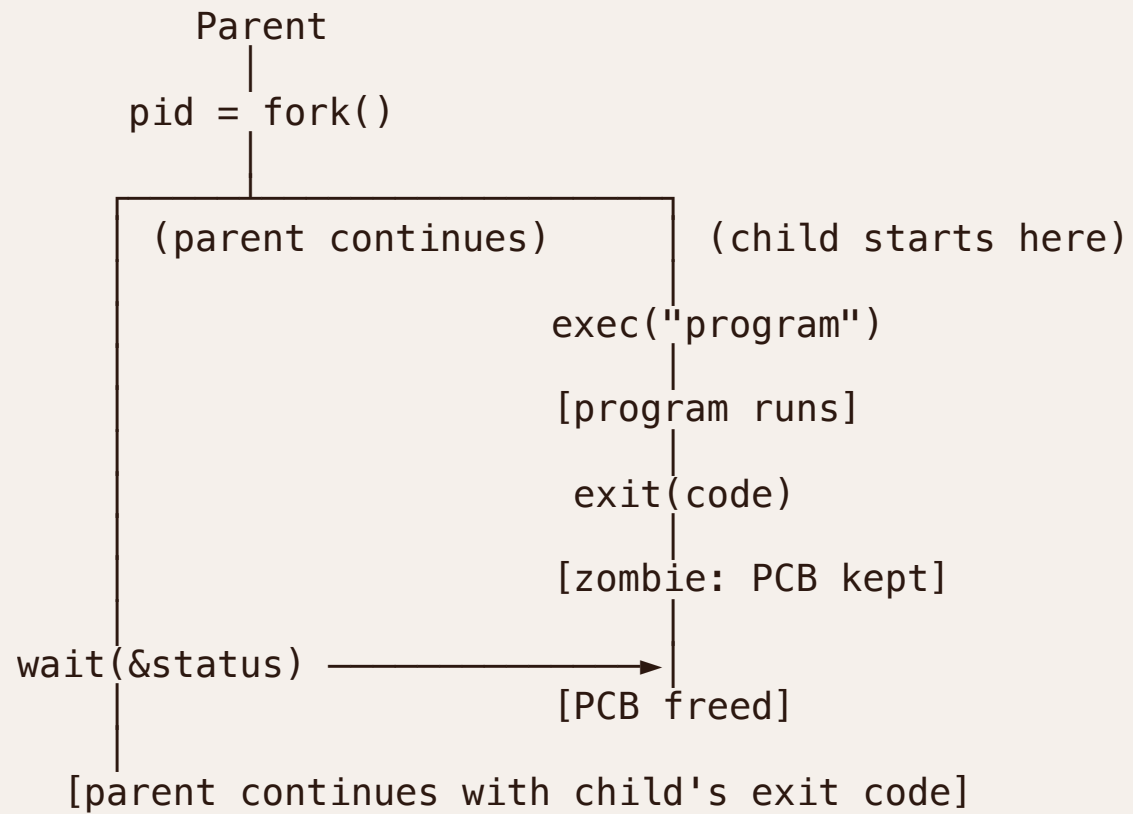
When a parent forks N children, it should wait for all of them:

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main(void) {
    int N = 5;
    for (int i = 0; i < N; i++) {
        pid_t pid = fork();
        if (pid == 0) {
            printf("Child %d (PID=%d) working...\n", i, getpid());
            sleep(i);    // each child sleeps a different amount
            exit(i);      // exit with its index as code
        }
    }
    // Parent waits for ALL children
    int status;
    pid_t pid;
    while ((pid = wait(&status)) > 0) {
        printf("Reaped child PID=%d, exit=%d\n", pid, WEXITSTATUS(status));
    }
    return 0;
}
```

| The Full Process Lifecycle

Putting `fork`, `exec`, `wait`, and `exit` together:



✓ Checkpoint 3 — `wait()` and `waitpid()`

Test your understanding:

1. What is a **zombie process**? How is it created, and how is it eliminated?
2. What does `wait(NULL)` do differently from `wait(&status)` ?
3. What does `WNOHANG` do in `waitpid()` ? In what scenario would you use it?
4. A parent forks 3 children. How many times should it call `wait()` to avoid zombies?
5. What is an **orphan process**? Who adopts it, and why is this not a problem?
6. When is `WIFSIGNALED(status)` true rather than `WIFEXITED(status)` ?

Trace the output of the multi-child `wait()` loop example before continuing.

§ 6 — `exec()`: Replacing the Process Image

The `exec()` Concept

`exec()` **replaces** the current process's memory image with a new program.

Before `exec()`:

```
my_program code  
my_program data  
my_program heap  
my_program stack
```

PID = 101

—exec→

After `exec()`:

```
ls code  
ls data  
ls heap (empty)  
ls stack
```

PID = 101 (same!)

Key insight: `exec()` does **not** create a new process. The PID stays the same. The old program is entirely replaced by the new one.

*If `exec()` succeeds, it **never returns** to the calling program. The calling code is gone.*

| The `exec()` Family

There are six variants — all ultimately call `execve()` :

Function	Args how?	File search?	Environment?
<code>execl(path, arg0, ..., NULL)</code>	List	No (full path)	Inherited
<code>execv(path, argv[])</code>	Array	No (full path)	Inherited
<code>execle(path, arg0, ..., NULL, envp[])</code>	List	No	Explicit
<code>execve(path, argv[], envp[])</code>	Array	No	Explicit
<code>execlp(file, arg0, ..., NULL)</code>	List	Yes (PATH)	Inherited
<code>execvp(file, argv[])</code>	Array	Yes (PATH)	Inherited

Most commonly used: `execvp()` — array form, PATH search, inherits environment.

execvp() – Parameters Explained

```
#include <unistd.h>

int execvp(const char *file, char *const argv[]);
//           ↑           ↑
//           program name argument array
//           (searched in PATH)
```

- **file**: the program name (e.g., **"ls"**) — searched in **\$PATH**.
- **argv[]**: a NULL-terminated array of strings:
 - **argv[0]** = program name (by convention)
 - **argv[1], argv[2], ...** = arguments
 - Last element **must** be **NULL**

```
char *args[] = { "ls", "-l", "-a", "/tmp", NULL };
//           ↑   ↑   ↑   ↑   ↑
//           argv[0] terminator
execvp("ls", args); // runs: ls -l -a /tmp
```

Simple `execvp()` Example

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int main(void) {
    printf("Before exec: PID=%d\n", getpid());

    char *args[] = { "ls", "-l", "/tmp", NULL };
    execvp("ls", args);

    // If we reach here, exec FAILED
    perror("execvp failed");
    exit(EXIT_FAILURE);
}

// Output:
// Before exec: PID=500
// total 48
// drwxrwxrwt 12 root root 4096 May 27 09:00 .
// ...
// (Note: "After exec" is never printed – the program was replaced)
```

What `exec()` Replaces vs. Preserves

Resource	After <code>exec()</code>
Code, data, heap, stack	Completely replaced by new program
PID	Preserved (same process)
PPID	Preserved
Open file descriptors	Preserved (unless <code>O_CLOEXEC</code> set)
Signal handlers	Reset to defaults
Signal mask	Preserved
Environment variables	Inherited (unless using <code>execve()</code>)
Process group, session	Preserved
Resource limits (<code>ulimit</code>)	Preserved

*The preservation of file descriptors across `exec()` is what enables **I/O redirection** in the shell.*

| Why `exec()` Never Returns on Success

When `exec()` succeeds, the kernel:

1. Loads the new ELF binary into the current process's address space.
2. **Overwrites** the code, data, heap, and stack segments.
3. Resets the PC to the new program's entry point (`_start`).
4. The return address that `exec()` would return to **no longer exists**.

```
execvp("ls", args);  
// The memory containing this line of code has been overwritten.  
// There is nowhere to return to.  
// The CPU now runs ls's code, not ours.  
  
perror("exec failed");    // only reached if exec() returned -1
```

Always put error-handling code **immediately after** the `exec()` call.

| The `fork()` + `exec()` Pattern

This is the **fundamental pattern** of process creation in UNIX:

```
pid_t pid = fork();

if (pid < 0) {
    perror("fork"); exit(1);
} else if (pid == 0) {
    // CHILD: replace our image with the target program
    char *args[] = { "wc", "-l", "file.txt", NULL };
    execvp("wc", args);
    perror("exec"); // only if exec fails
    exit(1);
} else {
    // PARENT: wait for child to finish
    int status;
    waitpid(pid, &status, 0);
    printf("wc finished with exit code %d\n", WEXITSTATUS(status));
}
```

The parent continues after the child runs the new program and exits.

| Why Separate `fork()` and `exec()` ?

This question is frequently asked. The answer lies in **flexibility**:

Between the `fork()` and `exec()`, the child can:

```
pid_t pid = fork();
if (pid == 0) {
    // 1. Redirect stdout to a file:
    int fd = open("output.txt", O_WRONLY|O_CREAT|O_TRUNC, 0644);
    dup2(fd, STDOUT_FILENO);
    close(fd);

    // 2. Set environment variables:
    setenv("MY_VAR", "hello", 1);

    // 3. Change working directory:
    chdir("/tmp");

    // 4. NOW run the program (with all the setup in place):
    execvp("ls", (char*[]){ "ls", NULL });
    exit(1);
}
```

A single `spawn()` would need to encode all this as parameters — intractable.

Environment Variables: `environ` and `execve()`

Every process inherits a set of `KEY=VALUE` strings as its **environment**:

```
#include <stdio.h>

extern char **environ;    // global pointer to environment array

int main(void) {
    for (char **e = environ; *e != NULL; e++) {
        printf("%s\n", *e);    // prints PATH=..., HOME=..., etc.
    }
    return 0;
}
```

`execve()` lets you provide a **custom environment** to the new program:

```
char *env[] = { "PATH=/bin:/usr/bin", "HOME=/tmp", "TERM=xterm", NULL };
char *args[] = { "env", NULL };
execve("/usr/bin/env", args, env);    // env will only see 3 variables
```

Building a Minimalist Shell

The shell is just a loop of: **read** → **fork** → **exec** → **wait**:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>

#define MAX_ARGS 64

int main(void) {
    char line[1024];
    while (1) {
        printf("mysh> ");
        fflush(stdout);
        if (!fgets(line, sizeof(line), stdin)) break;
        line[strcspn(line, "\n")] = '\0'; // strip newline

        // Tokenize
        char *args[MAX_ARGS];
        int n = 0;
        args[n++] = strtok(line, " ");
        while ((args[n++] = strtok(NULL, " ")) != NULL && n < MAX_ARGS);

        if (!args[0]) continue; // empty line
    }
}
```

Minimalist Shell — Continued

```
pid_t pid = fork();
if (pid < 0) {
    perror("fork"); continue;
} else if (pid == 0) {
    // Child: execute the command
    execvp(args[0], args);
    // If here, exec failed
    perror(args[0]);
    exit(EXIT_FAILURE);
} else {
    // Parent: wait for command to finish
    int status;
    waitpid(pid, &status, 0);
}
return 0;
}
```

This is a complete (though limited) shell. It handles any external command.

Real shells add: built-ins (`cd`, `exit`), redirection, pipes, job control.

✓ Checkpoint 4 — `exec()`

Test your understanding:

1. Does `exec()` create a new process? What does it actually do?
2. What is the PID of the process **after** a successful `execvp()` call?
3. What happens to open file descriptors when `exec()` is called?
4. Write the `argv[]` array to run: `grep -r "TODO" /src`
5. Why must the `argv[]` array be **NULL-terminated**?
6. Why is code placed **after** `execvp()` only reached when `exec` fails?
7. Explain in your own words why UNIX **separates** `fork()` and `exec()` rather than combining them into a single `spawn()`.

Compile and run the minimalist shell — add a `cd` built-in as a challenge.

§ 7 — Other Process APIs

`exit()` vs. `_exit()`

Two ways to terminate a process — with an important difference:

```
#include <stdlib.h>
void exit(int status);    // "clean" exit

#include <unistd.h>
void _exit(int status);   // "raw" exit
```

	<code>exit()</code>	<code>_exit()</code>
Flushes stdio buffers	✓ Yes	✗ No
Calls <code>atexit()</code> handlers	✓ Yes	✗ No
Closes open streams	✓ Yes	✗ No
Speed	Slower	Faster
Use in child after fork	⚠ Risk	✓ Preferred

In the child process after `fork()`, use `_exit()` to avoid flushing parent's stdio buffers twice. Or ensure you `exec()` before exiting.

| Signals — Asynchronous Events

Signals are software interrupts sent to a process:

```
#include <signal.h>
int kill(pid_t pid, int sig); // send signal 'sig' to process 'pid'
```

Common signals:

Signal	Number	Default Action	Cause
SIGTERM	15	Terminate	Polite termination request
SIGKILL	9	Terminate	Cannot be caught or ignored
SIGINT	2	Terminate	User pressed Ctrl+C
SIGCHLD	17	Ignored	Child changed state
SIGSEGV	11	Core dump	Invalid memory access
SIGSTOP	19	Stop	Cannot be caught or ignored

```
kill(child_pid, SIGTERM); // politely ask child to stop
kill(child_pid, SIGKILL); // forcefully kill child
```

Signal Handlers

A process can **catch** most signals and run custom code:

```
#include <signal.h>
#include <stdio.h>

void handle_sigint(int sig) {
    (void) sig;
    printf("\nCaught Ctrl+C (SIGINT). Cleaning up...\n");
    // do cleanup here
    exit(0);
}

int main(void) {
    signal(SIGINT, handle_sigint); // register handler

    printf("Running. Press Ctrl+C to stop.\n");
    while (1) {
        pause(); // sleep until a signal arrives
    }
    return 0;
}
```

SIGKILL (9) and **SIGSTOP** (19) **cannot** be caught, blocked, or ignored.

`nice()` – Process Priority

The OS scheduler uses a **nice**ness value (-20 to +19) to adjust CPU priority:

```
#include <unistd.h>
int nice(int inc);    // add 'inc' to current niceness
```

Niceness	Priority	Who can set it
-20	Highest	Root only
0	Default	Any user
+19	Lowest	Any user

```
# Shell usage:
nice -n 10 ./heavy_computation    # run with niceness +10 (lower priority)
renice -n -5 1234                  # change priority of PID 1234 (root only)
```

Higher niceness = "nicer to others" = lower CPU priority.

*Users can only **increase** their processes' niceness (be nicer), not decrease it.*

| pipe() — Inter-Process Communication

pipe() creates a **unidirectional communication channel** between processes:

```
int fd[2];
pipe(fd);
// fd[0] = read end
// fd[1] = write end
```

Classic pattern — shell `ls | wc -l`:

```
int fd[2];
pipe(fd);
if (fork() == 0) {
    // Child (producer): write to pipe
    close(fd[0]); // close unused read end
    dup2(fd[1], STDOUT_FILENO); // stdout → pipe write end
    execvp("ls", (char*[]){ "ls", NULL });
} else {
    // Parent (consumer): read from pipe
    close(fd[1]); // close unused write end
    dup2(fd[0], STDIN_FILENO); // stdin ← pipe read end
    execvp("wc", (char*[]){ "wc", "-l", NULL });
}
```

`dup2()` — I/O Redirection

`dup2(oldfd, newfd)` makes `newfd` point to the same open file as `oldfd`:

```
#include <unistd.h>
int dup2(int oldfd, int newfd);
```

Used to implement shell redirection (`ls > output.txt`):

```
// In the child, before exec:
int fd = open("output.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);
if (fd < 0) { perror("open"); exit(1); }

dup2(fd, STDOUT_FILENO); // fd 1 (stdout) now points to output.txt
close(fd);               // close the original fd (no longer needed)

execvp("ls", (char*[]){ "ls", "-l", NULL });
// ls thinks it's printing to stdout, but it goes to output.txt
```

The key insight: `exec()` preserves file descriptors, so the new program inherits the redirected stdout.

| The `/proc` Filesystem

Linux exposes live process information through the **virtual `/proc` filesystem**:

```
ls /proc/                # one directory per PID + system info
ls /proc/1234/           # info about process 1234

cat /proc/1234/status    # name, PID, PPID, state, memory
cat /proc/1234/cmdline   # command line arguments
cat /proc/1234/maps      # virtual memory map
cat /proc/1234/fd/       # open file descriptors (symlinks)
cat /proc/1234/environ   # environment variables

# Useful commands that read /proc:
ps aux                   # all processes
top                      # live process monitor
pmap 1234                # memory map of PID 1234
```

`/proc` is not a real filesystem — its files are generated on-the-fly by the kernel when read.

✓ Checkpoint 5 — Other Process APIs

Test your understanding:

1. Why should a child process call `_exit()` instead of `exit()` after `fork()` (without `exec()`)?
2. What is the difference between `SIGTERM` and `SIGKILL`? Which one can be caught?
3. What does `dup2(fd, 1)` do? What is the effect when followed by `exec()`?
4. A shell runs `ls > output.txt`. What sequence of syscalls achieves this?
5. What does `pipe(fd)` create, and what are `fd[0]` and `fd[1]`?
6. What information can you find in `/proc/1234/maps`?

Experiment: run `cat /proc/$$/status` in a terminal. What does `$$` give you?

§ 8 — The Shell: Putting It All Together

| How a Shell Works

A shell is fundamentally a **REPL** — Read-Eval-Print Loop:

Shell Main Loop

```
while (true) {  
    print_prompt();           // print "$ "  
    line = read_line();       // read user input  
    tokens = parse(line);     // split into words  
    if (is_builtin(tokens))  // cd, exit, etc.  
        run_builtin(tokens);  
    else {  
        pid = fork();  
        if (pid == 0) { setup_io(); execvp(...); }  
        else { wait_or_background(); }  
    }  
}
```

Shell Execution Model

User types: `ls -la | grep foo > results.txt`

Shell parses into:

Command 1: `["ls", "-la"]`

Pipe: `|`

Command 2: `["grep", "foo"]`

Redirect stdout: `> results.txt`

Shell creates pipe:

`pipe(fd)`

Shell forks command 1 (`ls`):

`stdout → fd[1] (pipe write end)`

`exec("ls", ["-la"])`

Shell forks command 2 (`grep`):

`stdin ← fd[0] (pipe read end)`

`stdout → open("results.txt")`

`exec("grep", ["foo"])`

Shell waits for both children.

Parsing User Input

A real shell tokenizes the command line carefully:

```
// Simple tokenizer (space-delimited, no quoting support):
char *tokenize(char *line, char **args, int max_args) {
    int n = 0;
    char *token = strtok(line, " \\t");
    while (token && n < max_args - 1) {
        args[n++] = token;
        token = strtok(NULL, " \\t");
    }
    args[n] = NULL;
    return NULL;
}

// Usage:
char line[] = "ls -la /tmp";
char *args[64];
tokenize(line, args, 64);
// args = { "ls", "-la", "/tmp", NULL }
```

Real shells (bash, zsh) handle quoting, escaping, variable expansion, globbing, etc.

I/O Redirection Implementation

How the shell implements `command > file`:

```
void redirect_output(const char *filename) {
    int fd = open(filename, O_WRONLY | O_CREAT | O_TRUNC, 0644);
    if (fd < 0) { perror("open"); exit(1); }
    dup2(fd, STDOUT_FILENO); // replace stdout with file
    close(fd);
}

// In the child process:
pid_t pid = fork();
if (pid == 0) {
    redirect_output("results.txt"); // set up redirection
    execvp(args[0], args);          // exec sees modified stdout
    perror(args[0]); exit(1);
}
waitpid(pid, NULL, 0);
```

Similarly, `< file` uses `dup2(fd, STDIN_FILENO)`.

Pipeline Implementation

How the shell implements `cmd1 | cmd2`:

```
int pipefd[2];
pipe(pipefd);

pid_t pid1 = fork();
if (pid1 == 0) {
    // cmd1: write to pipe
    close(pipefd[0]);                // close read end
    dup2(pipefd[1], STDOUT_FILENO); // stdout → pipe
    close(pipefd[1]);
    execvp(cmd1_args[0], cmd1_args);
    exit(1);
}
pid_t pid2 = fork();
if (pid2 == 0) {
    // cmd2: read from pipe
    close(pipefd[1]);                // close write end
    dup2(pipefd[0], STDIN_FILENO);   // stdin ← pipe
    close(pipefd[0]);
    execvp(cmd2_args[0], cmd2_args);
    exit(1);
}
close(pipefd[0]); close(pipefd[1]); // parent closes both ends
waitpid(pid1, NULL, 0);
waitpid(pid2, NULL, 0);
```

Background Jobs with &

When the user appends `&`, the shell runs the command in the background:

```
$ sleep 100 &  
[1] 1234      ← shell prints job number and PID, then returns prompt  
$           ← prompt is immediately available
```

Implementation — simply don't call `wait()`:

```
int background = (strcmp(args[n-1], "&") == 0);  
if (background) args[--n] = NULL; // strip '&' from args  
  
pid_t pid = fork();  
if (pid == 0) {  
    execvp(args[0], args);  
    exit(1);  
}  
// Foreground: wait for child  
if (!background) {  
    waitpid(pid, NULL, 0);  
} else {  
    printf("[1] %d\n", pid); // print job info and return  
    // child runs in background — will be reaped by SIGCHLD handler  
}
```

Shell Built-In Commands

Not all commands are external programs. **Built-ins** are handled by the shell itself:

Built-in	Why it must be a built-in
<code>cd</code>	Must change the shell's working directory (<code>chdir</code> in child wouldn't affect parent)
<code>exit</code>	Must terminate the shell process itself
<code>export</code>	Must modify the shell's own environment
<code>echo</code>	Could be external, but built-in is faster
<code>history</code>	Shell's own internal state

```
if (strcmp(args[0], "cd") == 0) {
    if (chdir(args[1] ? args[1] : getenv("HOME")) < 0)
        perror("cd");
} else if (strcmp(args[0], "exit") == 0) {
    exit(args[1] ? atoi(args[1]) : 0);
} else {
    // external command: fork + exec
}
```

Complete Shell — Full Trace

Tracing `grep -r TODO . | wc -l`:

1. Shell reads: `"grep -r TODO . | wc -l"`
2. Parses: `cmd1=["grep","-r","TODO","."]` `cmd2=["wc","-l"]`
3. `pipe(pipefd) → pipefd[0]=3, pipefd[1]=4`
4. `fork() → child A (PID 201)`
A: `close(3), dup2(4,1), close(4), execvp("grep",...)`
A runs `grep`; output goes to pipe
5. `fork() → child B (PID 202)`
B: `close(4), dup2(3,0), close(3), execvp("wc",...)`
B reads `grep`'s output; counts lines
6. Shell: `close(3), close(4)`
7. Shell: `waitpid(201,...) → waits for grep`
8. Shell: `waitpid(202,...) → waits for wc`
9. Shell prints prompt again.

✓ Checkpoint 6 — The Shell

Test your understanding:

1. Why must `cd` be a **shell built-in** rather than an external program?
2. What syscalls does the shell use to implement `ls > output.txt` ?
3. In a pipeline `A | B` , what happens to `pipefd[0]` and `pipefd[1]` in process A?
4. When the user types `sleep 100 &` , why doesn't the shell call `wait()` ?
5. Why must the shell **close both ends** of the pipe in the parent process after forking?
6. Describe the sequence of syscalls the shell uses to run a simple external command like `date` .

Challenge: add I/O redirection (`>`, `<`) and `&` support to the minimalist shell built in § 6.

§ 9 — Summary & Key Takeaways

| Process Abstraction — Recap

The **process** is the OS's fundamental unit of work:

- A process is a **running program** with its own: code, data, heap, stack, registers, file descriptors, and PID.
- The OS **time-shares** the CPU among processes, creating the illusion of parallelism.
- Processes move between **RUNNING** → **READY** → **BLOCKED** states based on scheduling and I/O.
- The OS tracks every process with a **PCB** (Process Control Block).
- Processes form a **hierarchy** (parent → child) rooted at PID 1.

Full API Reference

Syscall / Function	Purpose	Returns
<code>fork()</code>	Clone current process	<code>0</code> (child), <code>PID</code> (parent), <code>-1</code> (error)
<code>getpid()</code>	Get current PID	PID (always succeeds)
<code>getppid()</code>	Get parent PID	PPID (always succeeds)
<code>execvp(f, argv)</code>	Replace process image	Never returns on success; <code>-1</code> on error
<code>wait(&status)</code>	Wait for any child	Child PID, or <code>-1</code>
<code>waitpid(pid, &s, opt)</code>	Wait for specific child	Child PID, <code>0</code> , or <code>-1</code>
<code>exit(code)</code>	Terminate (flush buffers)	Never returns
<code>_exit(code)</code>	Terminate (raw)	Never returns
<code>kill(pid, sig)</code>	Send signal	<code>0</code> on success, <code>-1</code> on error
<code>pipe(fd)</code>	Create anonymous pipe	<code>0</code> on success, <code>-1</code> on error
<code>dup2(old, new)</code>	Duplicate file descriptor	<code>new</code> on success, <code>-1</code> on error

| Common Bugs and Pitfalls

Avoid these classic mistakes:

1. **Missing NULL terminator** in `argv[]` → undefined behavior in `exec()`.
2. **Zombie accumulation** — forgetting to call `wait()` after `fork()`.
3. **Double free / double flush** — calling `exit()` in child when parent has buffered stdio; use `_exit()`.
4. **Using `exec()` return value for success** — `exec()` only returns on failure.
5. **Not closing pipe ends** — child or parent holding a pipe end open prevents EOF detection.
6. **Assuming fixed fork ordering** — never write code that depends on parent or child running first.
7. **Not checking `fork()` / `exec()` return values** — always check for `-1` and call `perror()`.
8. **Dangling pointers in child** — modifying parent data structures in child before `exec()`.

The OS Role in Process Management

The OS is the **invisible conductor** behind every process:

User program calls `fork()`



▼ (syscall trap)

Kernel:

1. Allocates a new PCB
2. Copies parent's page table (COW)
3. Copies open file descriptor table
4. Assigns new PID
5. Sets child's `fork()` return = 0
6. Adds child to run queue
7. Returns child's PID to parent



▼ (return from syscall)

Both parent and child continue

The kernel enforces isolation, manages resources, and ensures fairness — transparently.

✓ Final Checkpoint — Lecture 2.1

Comprehensive review questions:

1. Trace the complete execution of: `fork(); if (fork() == 0) { fork(); }`. How many processes exist? Draw the tree.
2. A program forks a child. The child runs `execvp("ls", ...)`. What is the child's PID before and after `execvp`?
3. Write a C program that: forks 3 children, each printing its index (0, 1, 2), then the parent collects all exit codes and prints them in order.
4. Implement shell redirection: the child should run `ls -l` with stdout redirected to `out.txt`.
5. Explain why zombie processes occur and why they are only partially cleaned up without `wait()`.
6. Design a pipeline: `cat file.txt | sort | uniq -c`. How many forks and pipes does the shell need?

If you can answer all six, you are ready for Lecture 2.2.

| Lecture 2.1 Complete

Recommended Reading

- OSTEP Ch. 4: *The Abstraction: The Process*
- OSTEP Ch. 5: *Interlude: Process API*
- Available free at: <https://ostep.org>
- **Man Pages:** `fork`, `execve`, `waitpid`, `exit`, `kill`, `pipe`, `dup2`, `proc`, `signal`
- *The Linux Programming Interface* — Michael Kerrisk
- *Advanced Programming in the UNIX Environment* — Stevens & Rago

Contact & Q&A

- **Lecturer:** Gabriele Tolomei
- **Email:** tolomei@di.uniroma1.it
- **Office Hours:** Drop me a message, and we'll arrange a time to meet, either in person or online! 😊

Thank you for your attention!